Making GPUs go BRR

Horace He

If training loss lower than test loss
⇒ overfitting
⇒ Increasing capacity of model will not help

If training loss equal to validation loss
⇒ Wasting time regularizing model
⇒ Model is not overfitting
⇒ It generalizes equally well/poorly to unseen data
⇒ Model's capacity matches data complexity, & its not memorizing noise

3 components for deep learning perf

→ Compute: Time spent on GPU computing actual FLOPS

→ Memory: Time spent transfering tensors within a GPU

→ Overhead: Everything else

Know which of the above is a bottleneck before starting optimization.

spend all time doing transfers = Memory bound

when memory bound, increasing FLOPS will not help.

If majority time spent on FLOPS, rewriting code to C++/Rust to reduce overhead will not help.

Compute

→ Get max FLOPS, if possible.

→ Focus on maximizing compute, because the no of FLOPS in your algorithm can't really change without changing the actual ops being performed

→ Memory can be changed by just using more expensive ops

→ 'Compute' is a factory

Instructions ⇒ Overhead

Transfer raw materials ⇒ Memory

Efficiency ⇒ Compute

If factory efficiency ↑ faster than rate of material supply, we won't achieve peak efficiency (since we will be waiting for materials)

Tensor Cores ⇒ Specialized hardware for matmul

If doing ops apart from matmul, such as normalisation, exponentiation, etc, it will be slow.

Slow because majority time is spent transferring data for these ops i.e memory bandwidth bound.

Bandwidth

→ Moving from CPU to GPU, global CUDA mem to SRAM, etc.

Global CUDA to shared memory CUDA mem is called 'bandwidth cost'

Data transfer costs :→ } More relevant in distributed systems

Network costs :

Factory does all work, BUT not optimised for bulk storage.

Factory storage optimized for being fast (SRAM), & has less of it.

Where to store actual results & materials?

→ DRAM i.e warehouse

→ cheap & we have lots of space.

GPU DRAM stores up in nvidia-smi

For ops like torch.cos, majority of time is spent in moving data from warehouse to factory.
Hence, this is memory bound

Operator fusion ⇒ Perform multiple ops. Generally used when we have multiple ops on pointwise elements

custom CUDA kernels bring most benefit here.
Fusing ops reduces time. x.cos().cos() will take same time as x.cos()
Hence, many activation funcs are same cost.

Leads to implementation of recomputation (activation checkpointing) where instead of saving activations we just recompute them during backwards pass

A100 $\Rightarrow$ 1.5 TB PS

19.5 TFLOPS

32-bit floats (4 bytes)

$$\frac{375 \times 10^{12}}{4} \approx 400 \text{ billion.}$$

$$\frac{19.5 \times 10^{12}}{}$$

Increase compute = compute intensity.

Compute bound measurement

→ Measure achieved FLOPS as a percentage of peak FLOPS

Overhead

→ Python is slow. Launching kernel is overhead.

→ Increase size of data. If it doesn't increases runtime proportionally, you are overhead bound

→ Use Pytorch profiler.

Overhead bound → Tracing, op fusion, No Python, JIT

Bandwidth bound → Op fusion

Compute bound → Use Tensor cores, more $$$